



give a cheat code

consisting of all the commands to run the project and check everything there is to in the project as in npm start for frontend , eclipse for backend etc

all sql queries in one query

and all postman commands of all types of api testing

Here is your **project cheat code**: the exact commands and API calls to run the backend, frontend, database, and test every endpoint for your Library Management System. Spring Boot apps commonly run either from an IDE like Eclipse or via Maven from the terminal, React apps use `npm start`, and Postman testing for CRUD usually follows the GET/POST/PUT/DELETE REST pattern.^[1] ^[2]

Run everything

Backend in Eclipse

1. Open Eclipse / Spring Tool Suite.
2. Import the backend as an Existing Maven Project if not already imported.
3. Wait for dependencies to download.
4. Open the main class, usually:

```
LibrarymanagementApplication.java
```

5. Right click the file → Run As → Spring Boot App.

Backend in terminal

Go to the backend project folder and run:

```
mvn clean install  
mvn spring-boot:run
```

If you use the Maven wrapper:

```
./mvnw clean install  
./mvnw spring-boot:run
```

Your backend should run on:

```
http://localhost:8083
```

That port should match the `server.port=8083` value in your Spring Boot configuration.^[2]

Frontend in terminal

Go to the React frontend folder and run:

```
npm install  
npm start
```

If dependencies are already installed:

```
npm start
```

Frontend should open at:

```
http://localhost:3000
```

React development servers normally use port 3000 by default unless that port is occupied.^[3]
^[4]

SQL cheat code

Use this as one combined SQL setup script in MySQL Workbench or command line.

```
CREATE DATABASE IF NOT EXISTS library_management_system;  
USE library_management_system;  
  
SHOW DATABASES;  
SHOW TABLES;  
  
SELECT * FROM authors;  
SELECT * FROM books;  
SELECT * FROM user_profiles;  
SELECT * FROM user_profile_interests;  
  
DELETE FROM books;  
DELETE FROM authors;  
DELETE FROM user_profile_interests;  
DELETE FROM user_profiles;  
  
ALTER TABLE books AUTO_INCREMENT = 1;  
ALTER TABLE authors AUTO_INCREMENT = 1;  
ALTER TABLE user_profiles AUTO_INCREMENT = 1;  
  
DESC authors;  
DESC books;
```

```
DESC user_profiles;  
DESC user_profile_interests;
```

If Hibernate is creating tables automatically, you mainly need the database creation part first, and Spring Boot with JPA can generate the tables based on your entity classes and configuration.^[1]

Full project check list

Run these in this order:

1. Start MySQL.
2. Make sure the database exists.
3. Run Spring Boot backend in Eclipse or terminal.
4. Confirm backend starts on port 8083.
5. Run React frontend with `npm start`.
6. Open `http://localhost:3000`.
7. Test Authors page.
8. Test Books page.
9. Test Users page.
10. Test all APIs in Postman.
11. Check database tables with SQL `SELECT` queries.

This order reduces integration issues because Books depends on Authors, and the Users module depends on payload formatting for multiple field types.^[2] ^[1]

Postman cheat code

Base URL:

```
http://localhost:8083/api
```

Set header for POST and PUT requests:

```
Content-Type: application/json
```

Authors APIs

1. Add Author

POST

```
http://localhost:8083/api/authors
```

Body

```
{
  "authorName": "Chetan Bhagat",
  "email": "chetan@example.com",
  "biography": "Indian author and columnist."
}
```

2. Get All Authors

GET

```
http://localhost:8083/api/authors
```

3. Get Author By ID

GET

```
http://localhost:8083/api/authors/1
```

4. Update Author

PUT

```
http://localhost:8083/api/authors/1
```

Body

```
{
  "authorName": "Chetan Bhagat Updated",
  "email": "chetanupdated@example.com",
  "biography": "Updated biography of the author."
}
```

5. Delete Author

DELETE

```
http://localhost:8083/api/authors/1
```

Books APIs

1. Add Book

POST

```
http://localhost:8083/api/books
```

Body

```
{
  "title": "2 States",
  "isbn": "ISBN1001",
  "price": 499.0,
  "publishedDate": "2026-04-21",
  "genre": "Fiction",
  "availability": true,
  "language": "English",
  "pageCount": 320,
  "authorId": 1
}
```

2. Get All Books

GET

```
http://localhost:8083/api/books
```

3. Get Book By ID

GET

```
http://localhost:8083/api/books/1
```

4. Update Book

PUT

```
http://localhost:8083/api/books/1
```

Body

```
{
  "title": "2 States Updated",
  "isbn": "ISBN1001-NEW",
  "price": 550.0,
  "publishedDate": "2026-04-22",
  "genre": "Romance",
  "availability": true,
  "language": "English",
  "pageCount": 340,
  "authorId": 1
}
```

5. Delete Book

DELETE

```
http://localhost:8083/api/books/1
```

6. Search Books by Title

Use this only if your project already has a search endpoint.

GET

```
http://localhost:8083/api/books/search?title=2%20States
```

Users APIs

1. Add User

POST

```
http://localhost:8083/api/users
```

Body

```
{
  "fullName": "Rahul Sharma",
```

```
{
  "email": "rahul@example.com",
  "password": "Rahul@123",
  "gender": "Male",
  "interests": ["Reading", "Coding"],
  "role": "Student",
  "membershipDate": "2026-04-21",
  "loginTime": "2026-04-21T18:30:00",
  "age": 22,
  "profileFileName": "rahul-profile.png"
}
```

2. Get All Users

GET

```
http://localhost:8083/api/users
```

3. Get User By ID

GET

```
http://localhost:8083/api/users/1
```

4. Update User

PUT

```
http://localhost:8083/api/users/1
```

Body

```
{
  "fullName": "Rahul Sharma Updated",
  "email": "rahulupdated@example.com",
  "password": "Rahul@456",
  "gender": "Male",
  "interests": ["Music", "Travel"],
  "role": "Admin",
  "membershipDate": "2026-04-22",
  "loginTime": "2026-04-21T19:15:00",
  "age": 23,
  "profileFileName": "rahul-new.png"
}
```

5. Delete User

DELETE

```
http://localhost:8083/api/users/1
```

Your user payload is valid for the DTO you showed because it includes all required fields and uses a JSON array for `interests`, which maps naturally to a Java `List<String>` in Spring Boot. [\[1\]](#) [\[2\]](#)

Quick browser check

Open these after running both servers:

```
http://localhost:3000  
http://localhost:8083/api/authors  
http://localhost:8083/api/books  
http://localhost:8083/api/users
```

If the last three URLs return JSON in the browser, the backend is working. If the frontend loads and shows records, integration is working. [\[1\]](#)

Common fixes

Port 8083 not working

Check `application.properties`:

```
server.port=8083
```

React cannot call backend

Check the API base URL in your Axios/service file:

```
baseUrl: "http://localhost:8083/api"
```

Also keep CORS enabled for:

```
@CrossOrigin(origins = "http://localhost:3000")
```

CORS configuration is a standard requirement when a React frontend on port 3000 communicates with a Spring Boot backend on another port. [\[2\]](#) [\[1\]](#)

MySQL connection issue

Check your `application.properties`:

```
spring.datasource.url=jdbc:mysql://localhost:3306/library_management_system
spring.datasource.username=root
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

JPA with `ddl-auto=update` is commonly used in student CRUD projects so entity changes can sync to the schema during development.^[1]

npm start not working

Run:

```
npm install
npm start
```

If port 3000 is already in use, React usually prompts you to run on another port, but for your project it is better to close the other process and keep frontend on 3000 so CORS remains consistent.^[3]

Fastest final testing sequence

Copy this exact order into your notes:

1. Start MySQL
2. Create database
3. Run backend in Eclipse or `mvn spring-boot:run`
4. Check `http://localhost:8083/api/authors`
5. Run frontend using `npm start`
6. Open `http://localhost:3000`
7. Add author
8. Add book using existing `authorId`
9. Add user
10. Test edit and delete for all modules
11. Run SQL SELECT queries to verify data
12. Take screenshots

Optional terminal cheat block

```
# BACKEND
cd backend-folder
mvn clean install
mvn spring-boot:run

# FRONTEND
cd frontend-folder
```

```
npm install  
npm start
```

If you want, next I can give you this in an even more useful format:

- one-page viva cheat sheet
- one-page Postman cheat sheet
- one-page final submission checklist



1. <https://www.baeldung.com/spring-boot-react-crud>
2. <https://www.dhiwise.com/post/a-step-by-step-guide-to-implementing-react-spring-boot>
3. <https://namastedev.com/blog/creating-an-admin-dashboard-in-react-5/>
4. image.jpg